Case Study 1: AUTOSAR HLD Document Analysis Assistant

Project Overview: Automate the extraction, organization, search, and validation of architectural knowledge from large AUTOSAR High-Level Design documents.

1. Business Context

High-Level Design documents describe software components, interfaces, ports, signals, dependencies, functional flows, and integration logic. They are often hundreds of pages long and maintained as unstructured PDFs. Architects, developers, integrators, and testers must manually interpret this content. The lack of structured extraction and search makes it difficult to identify mismatches, missing dependencies, inconsistent terminology, and incomplete test coverage.

2. Project Objective

- Reduce manual effort required to review HLD documents.
- Create a structured and searchable repository of architecture knowledge.
- Improve component, interface, signal, and dependency traceability.
- Support design reviews, onboarding, impact analysis, and test planning.

3. Scope

In Scope: PDF and supported document ingestion; OCR for scanned pages; Component, interface, port, signal, and flow extraction; Natural language search with citations; Document comparison and inconsistency reporting; Export of structured findings.

Out of Scope: Automatic approval of architecture decisions; Modification of source HLD documents without review; Replacement of formal architecture governance.

4. Primary Users and Stakeholders

System architects, AUTOSAR architects, Software developers, Integration engineers, Test engineers, Engineering managers.

5. Proposed Solution

The proposed solution is a secure AI-assisted application using a locally or privately hosted Large Language Model, Retrieval-Augmented Generation, local vector storage, and a Python-based web application. The system supports engineers with evidence-based recommendations while retaining human review and approval.

6. Functional Capabilities

Document upload and controlled ingestion; Section, table, and metadata extraction; Architecture entity recognition; Interface and dependency mapping; Natural language question answering; Cited summaries and component reports; Revision comparison; Potential mismatch and completeness checks.

7. High-Level Workflow

1. Upload or select an approved HLD document.
2. Extract text, tables, headings, and metadata.
3. Chunk content by section and architecture context.
4. Create embeddings and save them in the local vector database.
5. Retrieve relevant sections for each user query.
6. Generate grounded responses with page or section citations.
7. Allow engineer review, correction, and export.

8. Inputs and Outputs

Inputs: AUTOSAR HLD documents, Interface specifications, Architecture descriptions, Component catalogues, Revision history.

Expected Outputs: Component and interface inventory, Dependency maps, Functional-flow summaries, Searchable Q&A; responses, Potential inconsistency report, Structured export for downstream use.

9. Suggested Technology Stack

User interface: Streamlit or Flask Web UI.
API and services: FastAPI preferred, Flask alternative.
LLM: Any approved LLM such as Llama, Mistral, Qwen, DeepSeek, Falcon, or equivalent, deployed locally or privately.
Embeddings: BGE, E5, Sentence Transformers, or equivalent.
Vector database: FAISS or ChromaDB, with Milvus or self-hosted Weaviate for scale.
RAG orchestration: LangChain, LlamaIndex, or a custom Python pipeline.
Structured storage: SQLite or PostgreSQL.
Deployment: Docker, optionally Kubernetes.

10. Expected Business Benefits

Reduced document-review effort; Faster architecture validation; Improved knowledge reuse; Better onboarding; Enhanced traceability to testing, safety, and cybersecurity.

11. Indicative Success Metrics

Document review effort reduction; Extraction precision and recall; Percentage of responses with valid citations; User adoption and repeat query volume; Number of confirmed mismatches or missing dependencies identified.

12. Key Risks and Controls

Incorrect extraction from tables or diagrams -> use page-level citations and review queues.
Hallucinated architecture relationships -> restrict answers to retrieved evidence and show confidence.
Unauthorized access to project documents -> apply role-based access and project isolation.
Outdated document versions -> enforce document version labels and approved-source status.

13. Implementation Approach

Phase 1 Discovery: collect representative HLDs, define taxonomy, questions, and success metrics.
Phase 2 Pilot: build ingestion, local vector store, RAG search, Streamlit UI, and export.
Phase 3 Validation: benchmark extraction and Q&A with architects and testers.
Phase 4 Scale: add revision comparison, graph relationships, access control, and integrations.

Governance Principle: AI-generated content must be traceable to approved source material and reviewed by authorized engineering specialists before it is used for compliance, design approval, software release, or vehicle validation.